MLFF Training-Data and Fine-Tuning Architecture

Purpose

This manual defines the accepted current scientific, statistical, execution, and evidence architecture for the mdstats MLFF training-data package. It covers source-certified atomistic data preparation, leakage-safe partitioning, target-data construction, MACE fine-tuning, evaluation, deployment verification, and campaign execution architecture.

The manual is state-oriented. It describes what mdstats **is**, not the sequence by which the implementation was developed. Proposed transitions and developer implementation gates belong under `workplans/`; completed release chronology belongs under `docs/history/mlff/`; correctness and performance evidence belong under `audits/`, `release/`, and `benchmarks/` as appropriate.

Architectural motive

MLFF campaigns mix several kinds of state that must not be conflated: physical source facts, eligibility decisions, statistical partitions, fitted transforms, subset-selection decisions, optimization/checkpoint state, evaluation evidence, and deployment decisions. The architecture therefore uses immutable, content-addressed records and explicit ownership boundaries. The same separation applies to execution: caches, schedulers, worker counts, memory layouts, and storage realization may change without silently changing scientific authority.

Expensive exact numerical work is computed once, exposed as independent work where safe, and reused downstream whenever its semantic inputs are unchanged. Exactness, deterministic authoritative reduction order, explicit resource ownership, and authenticated restart state take precedence over nominal utilization.

Canonical documentation layout

The release-facing authority is this assembled file and its synchronized PDF:

- `docs/arch_manuals/mlff_training_data_architecture.md`
- `docs/arch_manuals/mlff_training_data_architecture.pdf`

The maintainable source chapters live under `docs/arch_manuals/mlff_training_data/` and are assembled deterministically by `tools/build_mlff_architecture_manual.py`. Chapter files are retrieval units for the assembled authority and must not contradict it.

Detailed current behavior is owned by `docs/specs/training_data/`. Historical lineage is non-normative and is stored under `docs/history/mlff/`. Active implementation coordination is non-normative and stored under `workplans/active/`.

Reading index

Need	Primary chapter
Physical/statistical motivation and scope	Part I - Foundations and ownership
Source, labels, strain/stress, eligibility, feature/event contracts	Part II - Data and evidence contracts
Leakage control, cross-validation, selection, objective weighting	Part III - Statistical design and selection
Replay, MACE adapter, training/evaluation, active learning, determinism	Part IV - Training and evaluation
FEAS1/MVIDX1/MVSEL1/REPAIR1/MVQUAL1 theory and exact multi-view graph	Part V - Multi-view target-data architecture
Scheduler, exact execution, cache reuse, memory/storage, progress	Part VI - Performance and execution architecture
Cross-subsystem ownership and accepted design boundaries	Part VII - Ownership boundaries and decision summary
External scientific/algorithmic sources	References

Context retrieval index

For targeted human or AI loading, use the smallest authoritative source that contains the needed contract:

Query terms	Load first
`DATA*`, source/label identity, eligibility, stress/strain, features	`20_data_contracts.md`
partition, leakage, CV, selection, weighting, exposure	`30_statistical_design.md`
replay, MACE, checkpoint, evaluation, active learning, determinism	`40_training_evaluation.md`
FEAS1, MVIDX1, MVSEL1, REPAIR1, MVQUAL1, target rungs	`50_target_multiview.md`
scheduler, utilization, CSR/CSC, vectorization, memory, persistence, progress	`60_execution_performance.md`
ownership or accepted design boundary	`80_ownership_and_decisions.md`
provenance for an algorithmic/scientific idea	`90_references.md`
why/when a historical decision changed	`docs/history/mlff/`
a proposed implementation transition	`workplans/active/`

Normative vocabulary

- **SHALL / MUST**: required for scientific or execution correctness.
- **SHOULD**: default design unless measured evidence justifies another exact-equivalent realization.
- **MAY**: optional realization that cannot weaken scientific contracts.
- **authoritative evidence**: persisted information that defines or proves a scientific decision.
- **reconstructible execution cache**: discardable state derivable exactly from authoritative inputs.

Current release boundary

The current architecture uses exact multi-view target selection, deterministic resource-bounded CPU scheduling, authenticated sparse execution caches, restart-safe out-of-core MVIDX inversion, exact MVSEL-to-REPAIR state reuse before repair divergence, common fixed-width progress reporting, and bounded model-training/evaluation execution. Scientific identity and sequential decision authority are independent of worker count, queue completion order, memory layout, and reconstructible cache location.

Positive accelerator qualification that has not yet been executed is not architecture history or proof. Current release-qualification requirements remain in their owning specifications/runbooks; execution planning for unfinished qualification belongs in `workplans/active/`.

Part I - Foundations and ownership

Reader orientation

What an MLFF learns

An energy-conserving machine-learned force field represents a potential-energy function

$$E_\theta = E_\theta(\mathbf{Z}, \mathbf{R}, \mathbf{H}),$$

where $\mathbf{Z}$ contains atomic numbers, $\mathbf{R}$ contains positions, $\mathbf{H}$ is the periodic cell, and θ denotes model parameters. Forces and stress follow from derivatives of the same energy,

$$\mathbf{F}_i = -\frac{\partial E_\theta}{\partial \mathbf{R}_i}, \qquad \boldsymbol{\sigma} = -\frac{1}{V}\frac{\partial E_\theta}{\partial \boldsymbol{\epsilon}},$$

under the declared stress sign and strain convention of the label source. MACE constructs symmetry-aware local atomic features and sums atomic-energy contributions [1]. A useful training/evaluation corpus therefore constrains both the energy surface and its derivatives throughout the intended simulation domain.

A low average force error is not sufficient. Common framework vibrations can dominate aggregate statistics while rare mobile-ion environments, strain states, migration geometries, or other declared focus physics remain poorly represented. The architecture therefore separates broad numerical metrics, condition/group-resolved evidence, physical observable validation, and explicit extrapolation/challenge evidence.

Why adjacent MD frames are not independent

A molecular-dynamics trajectory contains temporally correlated configurations. Neighboring frames can be near duplicates, so placing them in different statistical roles can create leakage and overstate model accuracy.

For an observable x_t, the normalized autocorrelation at lag k is

$$\rho_x(k) = \frac{\langle (x_t - \bar{x})(x_{t+k} - \bar{x}) \rangle}{\langle (x_t - \bar{x})^2 \rangle}.$$

A truncated integrated autocorrelation time is

$$\tau_{\mathrm{int},x} = \Delta t \left[\frac{1}{2} + \sum_{k=1}^{k^\star} \rho_x(k) \right],$$

with an effective sample count approximately

$$N_{\mathrm{eff},x} \approx \frac{T}{2\tau_{\mathrm{int},x}}.$$

mdstats uses autocorrelation-aware complete-frame blocks, purge semantics, and explicit independence grades rather than treating every frame as an independent observation [3-5]. The precise estimator, truncation, block-size, purge, and role-assignment behavior belongs to the current sampling/partition specifications.

Statistical evidence roles

The architecture distinguishes gradient-training evidence from model-control and final-evaluation evidence.

Role	Function	May affect parameters?	May affect model/checkpoint choice?
Training/development	Supplies gradient updates and fitted training-domain products	Yes	Yes
Checkpoint monitor / validation	Controls declared stopping/checkpoint policy	No	Yes
Outer validation	Estimates protocol performance without fitting that protocol	No	No for the already-frozen job
Calibration	Calibrates final-committee uncertainty/acquisition behavior	No	No training/checkpoint change
Locked test / challenge	Final sealed evaluation of interpolation or named mechanisms	No	No

Calibration is not test data, and locked/challenge evidence is not ordinary validation data.

Scope and ownership

The MLFF training-data subsystem owns dataset-level certification, comparison, partition, selection, training-artifact construction, campaign orchestration, checkpoint/evaluation lineage, deployment verification coordination, and active-learning lineage.

Its current responsibilities include:

- VASP source discovery/certification and source/label identities;
- composition, thermodynamic condition, ensemble, reference-cell, and strain reconstruction;
- electronic-structure compatibility and label-domain grouping;
- energy/force/stress auditing and atomic-reference identifiability/fitting lineage;
- immutable frame facts, eligibility, and quality decisions;
- generic structural feature providers plus explicit optional material-profile extensions;
- event detection before ordinary thinning;
- autocorrelation-aware complete-frame blocks and role feasibility;
- fixed outer evidence roles and independent cross-validation job families;
- fold-local transforms, metrics, E0 fits, difficulty evidence, and selection;
- deterministic nested target-data construction and exact multi-view coverage/selection;
- MACE target/replay artifacts and explicit exposure realization;
- replay-retention monitoring and checkpoint admissibility;

- training/evaluation execution, protocol freeze, and committee export;
- final-committee-bound calibration and sealed evaluation activation;
- candidate admissibility/acquisition records and append-only active-learning lineage where supported by the current runtime/specification set.

LTA/zeolite ring, cage, site, crossing, and related semantics are optional profile extensions; they are not generic defaults.

The subsystem does not silently merge incompatible electronic-structure levels, infer ambiguous scientific references, use locked-test evidence for fitting/calibration/acquisition, treat replay-head disagreement as an uncertainty committee, redefine physical-analysis algorithms, or silently obtain external replay data.

Reference application: bulk Li/Na/K-LTA

The principal reference corpus contains 27 AIMD runs spanning seven cation compositions, three temperatures, and six additional LiNaK strain conditions. This application motivates, but does not hard-code into generic behavior, several design requirements:

1. framework atoms can outnumber mobile cations, so global descriptor averages must not hide declared mobile-species environments;
2. strain conditions need not form a full Cartesian product with composition and temperature, so condition schemas are hierarchical;
3. one trajectory per condition supplies limited independence and must not be represented as an independent-replica test;
4. fixed framework stoichiometry can make individual atomic reference-energy corrections non-identifiable without anchors;
5. short trajectories may contain few rare transitions, so absent events are explicit coverage gaps rather than evidence of irrelevance.

Relationship to existing mdstats capabilities

The training-data subsystem orchestrates existing mdstats scientific capabilities rather than duplicating them.

Existing capability	Reused evidence
`mdstats.io.vasp.read_vasp_frames`	cells, coordinates, energies, forces, stress, temperature, provenance
`mdstats.io.vasp_controls.read_vasp_run_controls`	source controls, named energy channels, SCF behavior
VASP ensemble-control certification	ensemble/control classification
trajectory-quality assessment	source and trajectory integrity verdicts
production-regime assessment	transient/stationary regime evidence
analysis structural/topology modules	optional profile-owned structural evidence
`mdstats.io.sampling_crossfit` and sampling primitives	source-bound block and purge semantics

Physical observables remain owned by `mdstats.analysis`; the MLFF layer may invoke and compare their results only through the declared analysis-owned contracts.

Controlling data flow

The current controlling flow is:

```
source bytes
  -> source occurrence identity
  -> controls + trajectory collection
  -> ensemble, quality, and production-regime evidence
  -> source catalog + decomposed label-domain audit
  -> structural atomic-reference identifiability
  -> immutable frame facts
       occurrence UID
       geometry fingerprint
       label payload digest
       labeled-configuration fingerprint
  -> labeled-frame eligibility
  -> full-resolution generic + partition-critical profile features
  -> event detection before ordinary thinning
  -> complete-frame temporal blocks
  -> fixed outer partition + independence evidence
       development pool
       outer monitor/validation
       calibration cohort when supported
       sealed interpolation/challenge tests when supported
  -> independent cross-validation job family
       fold-training domain
       disjoint checkpoint monitor
       held-out evaluation fold
       fold-local fitted products and selection
       fresh model/checkpoint per fold
       out-of-fold predictions
  -> final target-training fitted products + deterministic target-data order/rungs
  -> development MACE target/replay bundle
       no locked-test path
       replay-retention constraints
  -> candidate checkpoint evaluation and admissibility
  -> selected final checkpoints + independent-seed committee
  -> protocol freeze
  -> final-committee calibration where configured
  -> explicit sealed-evaluation activation
  -> deployment verification
  -> active-learning candidate/DFT lineage where configured
```

No allowed dependency runs from locked-test evidence into fitted transforms, E0 fitting, training selection, protocol/checkpoint choice, uncertainty calibration, or acquisition policy.

Package and responsibility structure

Current implementation is organized under source-independent sampling primitives, `mdstats.training_data` record/policy/workflow modules, optional feature/profile providers, MACE

export/runtime adapters, campaign orchestration, and analysis-owned observable bridges. The architectural requirement is responsibility separation rather than a frozen file listing: source facts, workflow decisions, fitted products, runtime realizations, and external-analysis results remain distinct owners even when modules are reorganized internally.

Public/serialized compatibility promises are controlled by current specifications and schema readers. Internal refactoring may reuse common sampling or execution primitives only when the externally owned scientific behavior and persisted identities remain compatible.

Part II - Data and evidence contracts

Evidence records and immutability

The MLFF data model separates source facts, workflow decisions, fitted products, runtime realizations, and external scientific results. A new policy creates new policy/decision records rather than mutating immutable source/frame facts.

Source and frame facts

`TrainingDataSource` owns source occurrence identity, path/location hints, content hashes, composition/controls, ensemble/quality/production evidence, label-domain identity, and declared reference grouping. `TrainingFrameRecord` owns source-bound frame facts such as `frame_uid`, source occurrence, frame index/time, atoms/cell, label references, conditions, and distinct geometry/label fingerprints.

`TrainingFrameRecord` does **not** own eligibility, partition, selection, exposure, calibration, or acquisition state.

Decision, policy, fitted, and realization records

Separate record families include, as applicable:

```
FrameEligibilityDecision
PartitionAssignment
SelectionAssignment
ExposureAssignment
CandidateAdmissibilityDecision
AcquisitionDecision

PartitionRoleBudgetPolicy
PartitionFeasibilityReport
FeatureMetricPolicyTemplate
FoldFeatureMetricFit
FinalFeatureMetricFit
SelectionBudgetPolicy
TrainingObjectivePolicy
ConfigurationWeightPolicy
PropertyWeightPolicy
CheckpointMetricPolicy
TrainingProtocolIdentity
MaceCheckpointControlPolicy
ExposureBackendPolicy
MaceExposureRealizationRecord
```

```
ReplayRetentionPolicy
ProtocolFreezeRecord
CalibrationApplicabilityDomain
CalibrationTransferDecision
```

A static policy defines an algorithm and fixed choices. A fitted record contains parameters learned from one explicitly authorized training domain. A realization record records behavior actually observed from an external/runtime system. Those roles are not interchangeable.

Digests and signatures

The architecture distinguishes deterministic content/policy/source digests from authenticated digital signatures. Content digests detect modification and bind identity but do not by themselves authenticate authorship. Serialized current records carry version/schema and deterministic content identity under their owning specifications.

Source manifest and occurrence identity

A review/production manifest supplies source locators, grouping declarations, scientific assertions that cannot be reconstructed unambiguously from one source file, and explicit expert overrides with rationale. Directory/file naming is diagnostic input, not accepted physical truth without verification.

The source byte/content identity is distinct from a manifest occurrence. Byte-identical copies may share a source-content identity while deliberately distinct manifest runs have distinct occurrence identities.

A frame occurrence is derived from the occurrence identity plus source frame index. This keeps occurrence identity stable across later concatenation/export while permitting duplicate-geometry detection across separate source occurrences.

Geometry, label, and labeled-configuration identities

The architecture keeps three identities separate:

```
geometry_fingerprint
label_payload_digest
labeled_configuration_fingerprint
```

`geometry_fingerprint` identifies atomic geometry independently of energy/force/stress labels under the current canonical wrapping/cell/tolerance policy. `label_payload_digest` binds the selected labeled payload and its label-domain identity. `labeled_configuration_fingerprint` combines geometry and label payload.

Leakage auditing uses occurrence overlap, exact geometry overlap, exact labeled-configuration overlap, declared near-geometry/descriptor criteria, restart/copy detection, and forbidden temporal proximity. Approximate/symmetry-aware matching may exist as an additional current policy only when explicitly specified; it cannot change the semantic roles above.

Electronic-structure label domains

Electronic-structure identity is decomposed because not every input difference has the same scientific meaning:

```
TheoryIdentity
EnergyReferenceIdentity
```

`DerivativeConvention`
`NumericalQualityProfile`
`SoftwareProvenance`

A versioned `LabelCompatibilityPolicy` classifies differences as compatible, compatible with quality evidence, separate label domain, or unresolved. Theory- or reference-defining differences cannot be silently waived.

A target training bundle contains one compatible target label domain and, when enabled, a separately identified replay head/lineage. Incompatible target DFT levels produce separate target bundles rather than an implicit mixed target domain.

Energy channel

The selected target energy is an explicit named channel consistent with the derivative labels. Its channel, units, completeness, electronic/reference convention, and provenance are preserved. Energy/force/stress labels that do not share an accepted derivative/reference convention are not silently combined.

Atomic-reference identifiability and fitting

For elemental correction vector $\Delta\mathbf{e}_0$, fitting has the schematic form

$$\mathbf{A}\,\Delta\mathbf{e}_0 \approx \mathbf{b},$$

with configuration-element count matrix $\mathbf{A}$ and target-minus-foundation energy residual $\mathbf{b}$.

Structural identifiability

`AtomicReferenceIdentifiabilityReport` depends on elemental count support rather than fitted target residuals. It records element order, matrix shape/rank/singular values, condition/null-space information, identifiable combinations, outcome, and transfer limitations. It does not contain fitted elemental corrections.

Rank deficiency can be acceptable only under an explicit fixed-domain/reference policy with its null space and transfer restrictions preserved. Fixed-stoichiometry systems must not imply individually identified elemental offsets when the count matrix does not support them.

Training-domain fit

`AtomicReferenceFitRecord` is a fitted object bound to one fold/final training domain, foundation checkpoint identity, identifiability report, solver/tolerance, elemental support, fitted corrections, residual, and policy outcome.

Each cross-validation fold receives its own fold-local fit. Final training receives a separate final-training fit. Outer monitors, calibration, held-out evaluation folds, and locked tests are excluded from the fit. Missing elemental support or new rank deficiency fails or invokes an explicitly declared alternative reference policy.

MACE export receives the exact accepted numerical E0 representation (normally an atomic-number mapping); a record/path name is provenance rather than an E0 payload.

Ensemble, temperature, cell, and strain

Ensemble and temperature

The subsystem consumes mdstats control/ensemble certification and distinguishes equilibrium, pressure-controlled, ramped/driven, multi-thermostat, and unresolved cases under the owning control specification. Ensemble is not inferred merely from observed cell variation.

Nominal temperature controls and realized ionic-temperature statistics remain separate. `TemperatureCondition` binds requested/thermostat targets, realized series/statistics, drift/stationarity evidence, and ramp status as applicable.

Reference-cell resolution

Strain requires an explicit or uniquely resolvable compatible reference. Accepted resolution order is controlled by current specifications and may use an explicit matrix/structure/run or a unique compatible unstrained member of the declared reference group. Ambiguity fails closed.

Cell and deformation convention

ASE cell vectors are rows. For fractional row vector $\mathbf s_{\rm row}$ and cell $\mathbf{H}$,

$$ \mathbf r_{\rm row}=\mathbf s_{\rm row}\mathbf H. $$

For reference $\mathbf{H}_0$ and current cell $\mathbf{H}_t$, the reported deformation gradient acting on Cartesian column vectors is

$$\mathbf{F}_t = \left(\mathbf{H}_0^{-1}\mathbf{H}_t\right)^T.$$

An internal right-acting row-vector form is acceptable only when serialization/reporting returns the declared Cartesian-column convention. Rotation/stretch separation uses the declared polar-decomposition convention. Stored strain evidence includes the applicable volume, linear/finite/logarithmic, hydrostatic/deviatoric, principal, shear, rotation, coordinate-frame, and storage-convention quantities.

Qualification includes nonsymmetric shear and rotated-stretch cases so transpose/left-right errors cannot hide behind diagonal fixtures.

Hierarchical condition schemas

Condition space is not assumed to be a global Cartesian product. A material profile declares applicable condition axes and hierarchical strata. The LTA profile, for example, separates unstrained composition/temperature/regime strata from strained composition/reference-condition/strain-mode/sign/regime strata. Only observed and scientifically applicable combinations are required.

Stress and virial

Canonical `REF_stress` is a symmetric Cartesian Cauchy-stress tensor in eV/Angstrom^3 under the ASE/MACE sign convention qualified by the runtime lock. Tensor shear carries no engineering-factor multiplication. Intermediate Voigt ordering, when used, is explicit and round-tripped.

Virial and stress have distinct keys and are never silently relabeled. Qualification covers units, finite-strain sign, tensor/Voigt order, shear factors, and MACE read-back. Missing stress may carry zero stress weight only under an explicit heterogeneous-label policy.

Eligibility and quality

Run/source quality distinguishes qualified, degraded, unqualified, and unresolved states under current policy. Overrides are explicit evidence.

`FrameEligibilityDecision` applies after labels exist. Hard rejection includes absent/nonfinite required labels or geometry/cell, singular/corrupt structures, incomplete ionic records not recoverable under the current trailing-interruption policy, catastrophic overlaps, and disallowed electronic-convergence failures. Soft evidence records transient regimes, unusual but physical forces/stress, rare coordination/events, topology changes, model residuals, and degraded numerical quality without turning percentile tails into automatic rejection.

Pre-DFT candidates use a separate `CandidateAdmissibilityDecision` over geometry/cell safety, element/count policy, topology/integrity, trajectory/integrator evidence, model outputs, and descriptor availability. After DFT labeling they re-enter normal source/frame eligibility lineage.

Material profiles and feature providers

Declarative profile boundary

`SystemProfileProvider` owns material identity: phases, geometry, chemistry modifiers, optional structural extensions, meaningful atom groups, condition axes, and independence axes. It does not itself own calculated scientific feature arrays.

Profiles are compositional rather than a single flat material enum. Interface/multiphase systems explicitly declare component membership. Generic fallback supplies only generic groups/axes; porous/zeolite/LTA semantics require the corresponding explicit extension chain and never activate automatically.

Universal structural selection features

The generic structural provider supplies selection-grade local geometry descriptors such as smooth chemistry-scaled coordination, support-neighbor count, radial projections, local-density/mixing proxies, angular moments, and rotationally invariant orientational-order summaries. These are frame/environment selection descriptors, not replacements for analysis-owned RDF, integer coordination, full angle distributions, structure factors, or topology observables.

Descriptors aggregate by authorized atom groups and elements present in the permitted domain. Generic temporal events capture large local structural changes without assigning material-specific physical meaning.

Partition-critical profile features

Rare categorical states that partition policy promises to protect are available at full resolution before the outer partition freezes. A profile may supply phase, environment, defect, region, molecular, or event states. Optional LTA state includes framework/mobile roles, resolvable ring/site class, off-center class, coordination/site/ring-crossing changes, and framework-integrity evidence.

Unresolved required classifications produce explicit coverage/partition limitations rather than fabricated balanced strata.

Optional profile extensions

Optional porous/zeolite/LTA providers may contribute framework/cation coordination, tetrahedral distortion/topology, ring/site/window geometry, site assignment, and transition/crossing evidence only when their extension is explicitly declared.

Optional learned-model features

A qualified optional MACE provider may supply foundation/model identity, invariant atomic descriptors, group/species environment summaries, and authorized zero-shot prediction/residual evidence. MACE/PyTorch remain optional dependencies to the mdstats core.

Feature blinding and fitted metrics

Geometry-only descriptors from a frozen model may be computed wherever authorized. Label-derived residual/difficulty features may be exposed only inside their applicable training domain.

Outer monitor, calibration, held-out evaluation, and locked-test residuals do not enter feature fitting or selection. Evaluation predictions can be persisted in blinded catalogs without exposing residual-derived selector inputs. Violating this boundary is a hard leakage failure.

Raw feature providers are partition-independent. Dataset-dependent scaling/PCA/whitening/metric fitting is represented by separate static templates and fold/final fitted records. For fold k, fitting may inspect only its gradient-training domain; other domains may be transformed by the frozen fitted object but cannot influence it.

A block-normalized metric may take the form

$$d^2(i,j) = \sum_b w_b \frac{\| \mathbf{z}_i^{(b)} - \mathbf{z}_j^{(b)} \|^2}{d_b},$$

where block weights, dimensions, missing-block behavior, dtype, tolerance, and any fitted scaling/projection are explicitly identified. High-dimensional learned descriptors do not dominate solely because they contain more components.

Event detection before thinning

The controlling order is:

1. source/label integrity on full-resolution frames;
2. event/change detection on all eligible frames;
3. protected event-window preservation;
4. temporal thinning of the ordinary non-event pool;
5. higher-cost descriptor/selection operations.

Event stencils are policy-controlled and compact by default. Adjacent frames from one physical event are not mistaken for independent rare-event evidence.

Autocorrelation and complete-frame blocks

Fast observables determine the minimum ordinary decorrelation/block scale; slow structural variables diagnose whether state-level independence exists at all. Candidate stride is defined in physical/frame time from the declared fast autocorrelation estimate and applies only to the non-event pool.

A `TrainingDataBlock` is a contiguous interval of whole configurations with block/run/frame bounds, represented time, regime, correlation evidence, and configuration identities. Atoms from one configuration are never split across statistical roles.

A purge interval separates roles using the declared physical/autocorrelation/event/restart policy. If a slow state never decorrelates, the independence report explicitly states that temporal blocking does not provide state-level independence.

Part III - Statistical design and selection

Outer partition architecture

Independence hierarchy

Evidence uses the strongest available independence level, for example:

1. independent replica/velocity seed or independently prepared realization;
2. independent structural/chemical ordering;
3. independent thermodynamic run;
4. purged temporal block within one run.

Temporal separation does not create an independent metastable state when the relevant slow variable never decorrelates. Every cohort carries machine-readable independence evidence and known limitations.

Partition-role feasibility

Before role assignment, `PartitionRoleBudgetPolicy` declares requested cohorts, cross-validation support, minimum independent blocks/grades, purge requirements, and allowed reductions. `PartitionFeasibilityReport` determines what the available evidence can actually support.

Outcomes include full support, temporal-block-only support, deferred calibration, external-only challenge evidence, reduced fold count, or insufficient support. The workflow never fabricates every desired role from a short trajectory merely to satisfy a percentage.

Outer evidence roles

A feasible target label domain may contain:

```
development_pool
outer_monitor_validation
uncertainty_calibration
locked_interpolation_test
zero or more locked_challenge_tests
```

Only the development pool supplies gradient-training candidates. The fixed outer monitor may control the current final-run monitoring/checkpoint policy but supplies no gradients and is not the locked test. Calibration is reserved for predictions from the actual final committee. Locked interpolation/challenge evidence cannot affect training, selection, checkpointing, calibration policy, acquisition policy, or protocol design.

When a requested role is unsupported, the role is absent/deferred with explicit evidence rather than synthesized from correlated data.

Independent cross-validation job families

A frame that has contributed a gradient is not independent validation evidence for that model. Likewise, a held-out evaluation fold cannot control stopping/checkpoint choice for the fold model whose error it is intended to estimate.

For K folds, job k contains distinct:

```
fold_training_domain_k
fold_checkpoint_monitor_k
held_out_evaluation_fold_k
```

The checkpoint monitor is a deterministic authorized split/cohort from non-evaluation evidence. The held-out evaluation fold remains inaccessible to fitted products and checkpoint choice.

The fold model has fresh model/optimizer/checkpoint lineage. Transform, feature-metric fit, E0 fit, difficulty evidence, and target selector are fitted only on the fold-training domain. Only after checkpoint choice freezes is the model evaluated on its held-out fold. The resulting out-of-fold catalog is bound to the complete `TrainingProtocolIdentity`.

Cross-validation is therefore a family of independent jobs, not a rotating epoch schedule inside one evolving model.

Training-set selection

Selection runs only inside the applicable fold-training or final-training domain. `SelectionBudgetPolicy` binds requested sizes, mandatory anchors/obligations, evidence-class budgets, near-duplicate policy, and deterministic tie/interleaving behavior.

Deterministic nested order

The selector constructs a deterministic ordered target-data sequence whose permitted dataset sizes are prefixes. Mandatory anchors/obligations are satisfied first; remaining capacity is allocated among the declared evidence classes without allowing one earlier class to consume the full budget.

Representative evidence classes include:

```
representative distribution coverage
species/atom-group environment coverage
rare/protected events
descriptor diversity/FPS
difficulty enrichment
```

Their exact fractions/counts are policy, not universal constants. Deficits are redistributed by the declared deterministic rule. A requested size smaller than mandatory support fails explicitly.

Hierarchical quotas

Every observed and scientifically applicable combination of declared condition axes/protected classes receives its policy-defined coverage request. Condition axes are profile-owned and may include composition, temperature, pressure, strain, phase, defect, surface/interface state, molecular conformer, or preparation history.

The optional LTA profile uses hierarchical unstrained and strained schemas rather than a global Cartesian product. Empty/non-applicable combinations are not treated as missing data.

Representative, diversity, and environment evidence

Representative anchors preserve dense expected-production regions; pure diversity sampling is insufficient because it can overweight feature-space boundaries.

Configuration-level farthest-point sampling uses the fitted heterogeneous feature metric with stable identity tie-breaking. It is one evidence source rather than the entire selector.

Declared focus atom groups receive separate environment coverage/selection so abundant host atoms cannot determine the entire target set. The generic architecture is group-driven; Li/Na/K groups are an LTA specialization, not core defaults.

Rare-event anchors

Protected event windows are retained around declared structural/chemical/trajectory changes. Generic changes include coordination/connectivity, large non-affine displacement, local packing/order changes, phase/state changes, strain extrema, and high but physical restoring-force excursions. Site/window/ring/interface/adsorption events activate only through the appropriate profile/provider.

Difficulty enrichment and blinding

Label-derived foundation-model residuals may enrich selection only inside the authorized training domain. Evaluation-domain residuals remain blinded. Difficulty enrichment is quota-controlled and cannot replace representative or hard coverage.

Coverage diagnostics

Selection evidence reports condition/group/feature coverage, nearest-distance/radius statistics, event/state counts, redundancy, and realized evidence-class budgets under the current target-data coverage authority. Coverage diagnostics recommend data sufficiency; they do not by themselves prove final model adequacy.

Training objective, weighting, and exposure

Training membership, label weighting, and runtime exposure are separate decisions.

`TrainingObjectivePolicy` binds loss family, energy/force/stress weights, head weights, normalization, robust-loss choices, and missing-label behavior. `ConfigurationWeightPolicy` and `PropertyWeightPolicy` bind condition/regime/event/quality and property-specific weights.

`ExposureAssignment`/realization evidence binds the head, eligible use, actual gradient exposures, configuration/property weights, sampling/duplication behavior, and seed/runtime lineage as applicable.

Atom-group force imbalance

A configuration can contain many more host force components than scientifically critical minority-group components. Selection diversity does not eliminate that loss imbalance.

The standard MACE configuration/property-weight path does not claim a generic atomwise group-weighted loss. Therefore evaluation/checkpoint policy reports declared group-resolved metrics and imposes applicable group constraints. Any custom atomwise/auxiliary loss defines a different `TrainingProtocolIdentity` and requires its own qualification.

Exposure backends

Exposure modes are distinct protocol semantics. The standard qualified fixed-file MACE path is `NATIVE_MACE_FIXED`: selected target/replay frames are materialized in fixed artifacts and the upstream loader performs the qualified shuffle/batching behavior.

Any custom epoch resampling, multi-job resampling, or final-refit behavior is valid only when a current adapter/specification explicitly supports it and binds its optimizer/checkpoint/exposure lineage. Static files alone cannot claim dynamic per-epoch resampling.

Realized MACE exposure

`MaceExposureRealizationRecord` compares intended artifacts/weights with observed loader behavior, including target/replay counts, implicit duplication, expected/observed batches, and configuration/property exposures.

Upstream target/replay duplication behavior is version-dependent. The adapter either disables unintended duplication when supported or binds the realized behavior into the protocol and verifies it. Silent changes in effective target/replay exposure fail closed.

Statistical authority boundary

The statistical design is controlled by explicit policies and immutable evidence lineage. Worker count, cache layout, training scheduler parallelism, and other execution realization cannot change partition membership, fold roles, selection order, checkpoint evidence role, or locked-test boundaries.

Part IV - Training and evaluation

Multi-head replay and training-protocol contract

Multi-head replay fine-tuning trains a shared MACE backbone on target data and a foundation replay dataset with separate output heads. The replay objective limits catastrophic forgetting while the target head adapts [11, 12]. Replay, objective, exposure, checkpoint control, backend, precision, optimizer/scheduler, and seed policy are part of the training protocol rather than incidental runtime settings.

`TrainingProtocolIdentity`

Every cross-validation family and final run is bound to one complete protocol identity containing, as applicable:

```
foundation checkpoint and head
model/foundation family and target head
naive or multi-head mode
replay source, selection, and monitor
training objective and property weights
target/replay head weights
exposure backend and realized balancing policy
checkpoint metric and checkpoint-control policy
replay-retention policy
optimizer, scheduler, epoch cap, stopping/LR policy, and seed policy
model precision and execution backend
MACE adapter/runtime lock
```

Cross-validation results apply only to that identity. Results from a different replay mode, objective, precision/backend realization, checkpoint policy, or other protocol-defining choice are not validation of the final protocol.

Separate target and replay lineages

Target and replay evidence retain separate source/label identities, atomic-reference policy where applicable, selection/split plans, weights/exposure accounting, and validation/sentinel monitoring. Replay train and replay monitor roles are disjoint.

The mdstats core records replay preparation and does not silently download external replay data. True-label replay is evaluated against held-out labels; pseudo-label replay measures drift from the bound foundation model on an unseen sentinel set.

Replay retention

`ReplayRetentionPolicy` binds the retention metric, foundation/pre-fine-tuning baseline, tolerated degradation, aggregation over properties, and failure/override behavior. Candidate checkpoints that violate a mandatory replay-retention constraint are inadmissible even when target error improves.

Checkpoint metrics and constrained choice

`CheckpointMetricPolicy` defines the primary target objective together with all mandatory target, focus-group/species, condition, stress/property, and replay constraints. Candidate checkpoint selection is deterministic over the complete evaluated candidate set and fails closed when no candidate satisfies mandatory constraints.

A typical mathematical form is

$$\min_{c} L_{\text{target monitor}}(c)$$

subject to profile- and protocol-specific constraints such as

$$L_{F,g}(c) \leq \delta_g, \qquad \Delta L_{\text{replay}}(c) \leq \delta_{\text{replay}}.$$

Exact metrics and thresholds are serialized policy, not hard-coded universal constants.

MACE checkpoint control

The supported MACE adapter is version-locked and verifies the native validation-head ordering, scheduling/stopping behavior, checkpoint retention, target/replay loader realization, and other upstream behaviors on which the protocol depends. The accepted native-target-monitor mode ensures that the target checkpoint monitor owns native scheduling/checkpoint control while replay behavior cannot silently terminate the run.

Every candidate checkpoint needed by the external selection policy is retained and evaluated on the authorized target and replay monitors. If the version-locked upstream behavior changes, preparation/qualification fails closed rather than silently accepting a different control flow.

MACE adapter and artifact boundary

Version/runtime lock

Every supported runtime lock records sufficient identity to reproduce and requalify upstream-dependent behavior, including package/source identity, relevant CLI/parser/loader/train-loop identity,

validated head order, checkpoint behavior, replay-ratio behavior, precision/backend realization, and accelerator qualification where applicable. Documentation URLs alone are not treated as a stable API contract.

Minimal Extended XYZ plus sidecar provenance
Extended XYZ contains only MACE-readable labels, weights, and compact stable identities. Long provenance, policy identities, and selection/audit reasons live in a sidecar manifest keyed by `frame_uid`.

Target-frame export includes the declared energy channel, forces, stress when available/authorized, stable frame/config identities, configuration/property weights, cell/PBC, atom order, and exact label-domain/E0 provenance. Export uses sufficient numerical precision and certifies round-trip semantics through the locked parser/reader path.

Separated development, calibration, and sealed-evaluation artifacts
The architecture separates:

```
development_bundle/
calibration_bundle/
sealed_evaluation_bundle/
evaluation_activation/
evaluation_results/
```

Development artifacts contain no locked-test path. A sealed evaluation bundle may exist before activation, but training and checkpoint selection cannot inspect it. Activation requires the applicable `ProtocolFreezeRecord`, selected committee identity, complete training-protocol identity, checkpoint-selection decision, and other owning-specification predicates.

Explicit E0 serialization
`AtomicReferenceFitRecord` is converted to the exact upstream representation accepted by the runtime lock, normally an explicit atomic-number mapping. A conceptual record name/path is provenance and is never substituted for the numerical `E0s` payload.

One compatible target label domain per bundle
A target bundle contains one compatible target `LabelDomain` and, when replay is enabled, its separately identified replay head/lineage. Incompatible target electronic-structure domains are not silently merged.

Export/loader qualification
The adapter qualifies atom order, cell/PBC, selected energy, forces, stress/virial convention, weights, head labels/order, explicit E0 mapping, parser recognition, effective target/replay counts, and downstream element mapping where required. Intended exposure never substitutes for observed loader realization.

Protocol-matched cross-validation and final training

The current workflow preserves a strict dependency order:

1. freeze one outer partition, feasibility report, and independence evidence;

2. bind each candidate protocol to complete `TrainingProtocolIdentity` and replay/exposure/checkpoint lineages;
3. create independent cross-validation jobs with fold-training, disjoint checkpoint-monitor, and held-out evaluation domains;
4. fit transforms, metrics, E0, difficulty evidence, and target selection using only each fold-training domain;
5. train a fresh model for each fold under the bound checkpoint-control policy;
6. freeze checkpoint choice without inspecting the held-out evaluation fold;
7. evaluate the frozen checkpoint on the held-out fold and aggregate protocol-matched out-of-fold evidence;
8. freeze the chosen protocol/data/selection/stopping/checkpoint/seed policies;
9. fit final training-domain products and train the declared independent final seeds;
10. externally evaluate/admit candidate checkpoints, export the selected target heads, and construct the final committee;
11. emit protocol/committee freeze evidence;
12. calibrate final-committee uncertainty on a dedicated authorized cohort where available;
13. activate sealed evaluation only after all promotion predicates pass;
14. execute bounded deployment verification under the frozen model/runtime identity.

If a protocol intentionally consumes an ordinary monitor during final refit, that loss of independent monitoring is explicit in the protocol/evidence lineage; it cannot be hidden by relabeling the consumed data.

Training monitoring, stopping, and learning-rate control

Online monitors are deterministic, common protocol inputs rather than resampled per epoch. Lightweight monitoring may control target-oriented stopping or detect unacceptable replay degradation only under the current stopping specification. The held-out cross-validation evaluation fold never controls stopping or checkpoint choice.

Learning-rate scheduling/refinement is part of `TrainingProtocolIdentity`. Scheduler changes, epoch-cap changes, or checkpoint-control changes define a different protocol and require protocol-matched validation rather than being applied after comparison.

Evaluation and candidate reduction

Checkpoint evaluation proceeds from lightweight online evidence to the current bounded full-evaluation/selection policy without changing the role of the underlying evidence. Screening reduces computation; it does not authorize inspecting locked-test data or changing thresholds after seeing candidate results.

Full candidate metrics are persisted with their exact model/data/runtime identities. Replay retention is a hard admissibility condition rather than a bonus in a combined target score unless the current metric policy explicitly says otherwise. Where physical relaxation/deployment integrity is required, structural failure is a rejection condition independent of numerical force-RMSE ranking.

Committee, protocol freeze, and sealed evaluation

A committee is constructed only from selected final-run target heads with explicit seed/member identity. `ProtocolFreezeRecord` binds the selected training protocol, model/checkpoint identities, committee identity, and required upstream evidence.

Locked interpolation/challenge evaluation is operationally sealed until the applicable activation predicates pass. Locked evidence cannot retroactively alter training selection, stopping, checkpoint choice, replay policy, calibration policy, or acquisition rules.

Calibration and active-learning lineage

Committee disagreement is a ranking signal rather than an error guarantee [13, 14]. Numerical uncertainty/acquisition thresholds are calibrated only from predictions of the actual frozen final committee on an authorized calibration cohort.

Calibration identity binds the committee/model digests, training/replay/seed/runtime lineage, precision/backend, calibration cohort, and declared applicability domain. The applicability domain records relevant elements/compositions, thermodynamic/strain ranges, cell sizes, structural/event classes, descriptor-distance ranges, force/stress ranges, and integrity states.

`CalibrationTransferDecision` distinguishes at least:

```
within_calibrated_domain
rank_only_outside_domain
recalibration_required
rejected_incompatible_domain
```

Without valid final-committee calibration, acquisition is explicitly uncalibrated/rank-only. Locked tests are excluded from calibration and acquisition.

Selection-biased active-learning labels enter a new development/training candidate pool. Existing frame roles are inherited unchanged by default. Independent new evidence may create new calibration/validation/challenge cohorts only under explicit lineage. Repartitioning existing evidence creates a new evaluation lineage rather than silently rewriting the old one.

Determinism and reproducibility

A reproducible campaign binds source and parser identities, policies/digests, reference-cell/deformation conventions, feature providers, foundation/model/runtime locks, random seeds/dtype/backend, fitted metrics/E0, partition/independence evidence, target selection/coverage policy, training objective/weights, complete protocol identity, exposure realization, replay-retention and checkpoint decisions, committee/protocol freeze, activation/calibration evidence, role inheritance, tie rules, fold assignments, ordered selections, and output checksums as applicable.

Execution-only worker counts, queue completion order, cache location, and storage layout are excluded from scientific identities unless a current specification explicitly declares otherwise.

Failure semantics

The workflow fails closed when, among other owning-specification conditions:

- source/label identity is unresolved or required labels are invalid;
- incompatible label domains are mixed;

- strain/reference conventions are ambiguous;
- requested evidence roles are infeasible under the declared independence policy;
- monitor/locked evidence reaches a forbidden fitted/selection/calibration/acquisition operation;
- a held-out evaluation fold controls checkpoint choice;
- cross-validation and final training do not share the compared complete protocol identity;
- required MACE/runtime behavior differs from its qualified lock;
- realized target/replay exposure differs from the accepted plan;
- a locked-test path appears in development configuration;
- no checkpoint satisfies mandatory target/focus/replay/integrity constraints;
- replay checkpoint/source lineage is incompatible;
- calibrated acquisition is attempted outside its applicability domain without the declared transfer action;
- active-learning child generation rewrites existing roles without a new evaluation lineage.

Absent rare events, replicas, condition combinations, calibration cohorts, or challenge sets are reported as limitations/coverage gaps rather than fabricated evidence.

Part V - Multi-view target-data architecture

Motivation and authority

A target-data subset must cover several physically meaningful feature views simultaneously. Optimizing only an average distance or one descriptor can hide a severe deficit in another required view. The multi-view architecture treats each required family as an explicit coverage constraint, diagnoses full-pool feasibility before subset optimization, and preserves deterministic nested target sets so size/fidelity comparisons are not confounded by resampling.

The architecture follows four rules:

1. feasibility precedes subset optimization;
2. hard coverage cannot be traded for aggregate utility;
3. redundancy is defined through unique covered witness mass and hard/provenance obligations rather than local density alone;
4. selector state and independent qualification remain separate authorities.

Exact neighborhood graph

For feature family m, let $x_w^{(m)}$ be witness coordinates, $x_c^{(m)}$ candidate coordinates, D_m the frozen scaling transform, and $r_w^{(m)}$ the authoritative witness radius. Exact adjacency is

$$A_{wc}^{(m)} = \left[\left\| D_m \left(x_w^{(m)} - x_c^{(m)} \right) \right\|_2 \le r_w^{(m)} \right].$$

Production authority uses exact radius semantics; approximate-neighbor substitutions are not scientifically equivalent unless a future accepted specification explicitly changes that contract.

For selected subset S,

$$n_w^{(m)}(S) = \sum_{c \in S} A_{wc}^{(m)},$$

and weighted family coverage is

$$C_m(S) = \frac{\sum_w \omega_w^{(m)} \mathbf{1}[n_w^{(m)}(S) > 0]}{\sum_w \omega_w^{(m)}}.$$

For hard threshold τ defined by the current coverage policy, robust deficit is

$$D_{\max}(S) = \max_m \max(0, \tau - C_m(S)).$$

A weighted average cannot substitute for a failed required view.

FEAS1 - full-pool feasibility and fragility

FEAS1 evaluates the complete eligible candidate/reference authority before subset optimization. It verifies expected self/cross support, measures low-support fragility, records candidate-degree/support evidence, and derives conservative lower bounds needed to satisfy hard support/obligation constraints.

For witness w,

$$d_w^{(m)} = \sum_{c \in \mathcal{C}} A_{wc}^{(m)}.$$

Low-degree witness mass identifies regions where correlation-unit exclusion or subset restriction can destroy support. A capacity diagnosis is evidence that a requested ceiling/rung cannot satisfy the frozen predicates; it is not permission to relax those predicates silently.

MVIDX1 - one shared exact sparse relation

MVIDX1 reuses the exact neighborhood relation already produced/qualified for the same semantic inputs. FEAS1 and MVIDX are therefore consumers of one exact neighborhood authority rather than independent geometric implementations.

Canonical sparse execution uses witness-oriented CSR-equivalent storage with fixed typed offsets/indices and FP64 scientific weights stored separately. Identity binds candidate/reference ordering, family/scaling/radius/distance semantics, cardinalities, and cache/schema version; execution-only worker/block/queue/storage choices are excluded.

MVIDX adopts authenticated forward witness-to-candidate CSR and constructs candidate-to-witness inverse state without repeating geometry. Forward/inverse edge cardinality and identities are cross-checked exactly.

Large inversions may use the current deterministic out-of-core implementation described in Part VI, but in-memory and file-backed realizations remain byte-equivalent for authoritative sparse arrays.

MVSEL1 - deterministic progressive selection

MVSEL constructs one global selection order whose permitted target sets are prefixes/rungs defined by the current target-data policy. Mandatory reservations and unsatisfied hard views/strata are serviced before discretionary representative filling.

At each rank, admissible candidates are compared by the frozen lexicographic priorities, including hard/worst-view deficit reduction, newly covered weighted mass, correlation/provenance balance, representative gain, normalized diversity, and stable candidate identity as applicable.

Rank authority is sequential because selection state changes after every accepted candidate. Parallel/vector execution may accelerate exact sparse state preparation/mutation only when the authoritative candidate choice and FP state remain equivalent.

The selector maintains witness multiplicity, hard-obligation state, and candidate marginal state incrementally through inverse adjacency. Full candidate-by-witness rescoring after each rank is not the current execution architecture.

REPAIR1 - exact shell repair

For selected candidate c, unique covered mass follows from multiplicity-one witnesses:

$$U(c \mid S) = \sum_{m} \sum_{w : A^{(m)}_{wc} = 1} \omega^{(m)}_{w} \, \mathbf{1}[n^{(m)}_{w}(S) = 1].$$

Removal candidates must have sufficiently small/allowed unique contribution and no unique mandatory obligation. Replacement candidates come from the declared deficit/frontier policy. Every accepted swap obeys the frozen objective/tie hierarchy and preserves lower protected prefixes/rungs.

Proposal scoring within one immutable pre-swap state may execute concurrently, but accepted-winner comparison and authoritative state mutation remain deterministic. Exact selector-to-repair state reuse is governed by Part VI: a pure-selector checkpoint is valid only before repair divergence.

MVQUAL1 - independent same-N qualification

MVQUAL independently recomputes coverage/obligation evidence for candidate subsets at identical cardinality. It records the current hard-view deficits, uncovered mass/count, redundancy/unique-support evidence, provenance/correlation diversity, and other policy-defined diagnostics.

Selector-internal counters are not accepted as independent qualification evidence. Qualification may share authenticated primitive sparse inputs but recomputes the relevant predicates through its own verification path. Locked-test data cannot tune radii, weights, repair budgets, tie rules, or qualification thresholds.

Target-size and fidelity funnel

The allowed nested sizes and screening/fidelity stages are current specification/policy, not architecture chronology. Architecture requires:

- a deterministic ordered/rung family whose smaller accepted sets are protected prefixes of larger ones where the current policy declares nesting;
- a hard coverage/obligation feasibility screen before expensive training;
- deterministic reduction of surviving candidate sizes under the declared zero-shot/short/full training policy;
- the smaller-size tie preference whenever the current indistinguishability criterion is satisfied;
- fail-closed behavior when too few sizes satisfy the minimum coverage/feasibility requirement;
- full-fidelity comparison only among survivors authorized by the earlier current-policy stages.

The exact size list, epoch budgets, indistinguishability threshold, survivor counts, and coverage threshold belong to the current target-data specifications/policies and are not duplicated here as a developer roadmap.

Scientific non-negotiables

Execution optimization does not authorize approximate neighborhood search, relaxed hard coverage, changing correlation/leakage boundaries, altering sequential selection/repair decision authority, or using locked evidence to tune target-data policy. Any scientific change to those semantics requires an explicit specification/architecture revision rather than an execution optimization.

Part VI - Performance and execution architecture

Performance objective and authority

Execution optimization is accepted only when it preserves scientific authority and improves measured throughput, memory behavior, or restart cost. CPU/GPU utilization is diagnostic rather than scientific authority. Memory-bound sparse kernels may be optimal below nominal occupancy targets.

For a stage allocated P CPU lanes, effective occupancy over a bulk interval is

$$U_P = \frac{\Delta t_{\mathrm{CPU}}}{P\,\Delta t_{\mathrm{wall}}}.$$

When sufficient independent compute tasks exist and the kernel is compute-bound, automatic execution targets high sustained occupancy while respecting the configured CPU/RAM/GPU/VRAM ceilings. Throughput and wall time decide among exact-equivalent realizations; scientific digests and authoritative records decide correctness.

Worker count, queue depth, query-block size, storage path, cache location, and other execution-only choices SHALL NOT enter scientific identity unless the value changes a declared scientific algorithm.

Work/span model and single-level parallelism

The campaign follows a task-parallel work/span model. With serial work T_1 and critical path T_∞,

$$T_P \geq \max\left(\frac{T_1}{P}, T_\infty\right).$$

Independent work is exposed at the highest level that supplies enough tasks. Nested numerical parallelism is suppressed while the outer queue can fill the budget:

$$P_{\mathrm{outer}} \times P_{\mathrm{native}} \leq P_{\mathrm{budget}}.$$

For cKDTree, BLAS, OpenMP, and similar native kernels, campaign execution normally uses one native lane per task when outer work is sufficient. Native-thread configuration is owned by the stage/resource scope rather than toggled independently inside arbitrary workers.

Shared deterministic CPU scheduler

`DeterministicWorkQueue` is the common substrate for CPU-heavy independent work. Its current execution contract provides:

- explicit `StageResourceScope` CPU and RAM ownership;
- separately bounded ready, submitted/in-flight, and completed work;
- work-conserving dispatch across compatible profiles, families, domains, or jobs;
- deterministic task identities and exception propagation;

- deterministic ordered reducers where authoritative FP64 reduction order matters;
- memory-weighted admission/backpressure and explicit persistent-memory reservations;
- queue/executor heartbeat telemetry;
- locality metadata that does not enter scientific identity.

The executor owns exactly the executing Python lanes admitted by the resource scope. It MAY retain more submitted futures than executing lanes to hide coordinator hand-off latency, but simultaneously executing work remains bounded by the resource scope and does not authorize nested oversubscription.

Task completion may be arbitrary. Whenever arithmetic order is part of exact-equivalence authority, authoritative state is committed only in the prescribed canonical order.

Bare library/API calls that do not receive an explicit campaign scope preserve their documented direct-call resource semantics. Campaign orchestration supplies the explicit bounded scope and therefore owns admission, native-thread quarantine, and hard resource ceilings.

Exact neighborhood production and reuse

`ExactNeighborhoodEngine` is the single exact TARGET-DATA2B/C geometric neighborhood implementation. Query blocks from eligible feature families may execute through the shared deterministic queue. The frozen scaled-distance/radius/tolerance semantics and candidate/witness order remain scientific authority.

Completed blocks are reduced in canonical witness order and streamed into authenticated witness-oriented CSR state. Ragged neighbor temporaries are released after canonical commit. The final CSR uses fixed typed offsets/indices and is admitted against the stage RAM budget before materialization.

The neighborhood store is reconstructible execution state. Its identity binds the semantic reference/candidate ordering, family identity, metric/tolerance policy, cardinalities, and cache-format version. Worker count, block size, queue depth, timing, and progress configuration are excluded.

MVIDX consumes authenticated forward CSR and SHALL NOT perform a second geometric query on a cache hit. Missing, corrupt, or stale forward state is rebuilt through the same exact neighborhood engine rather than a separate geometry implementation.

Deterministic MVIDX inversion and out-of-core scaling

Required-family candidate-to-witness inversion and hard-obligation inversion are independent exact tasks. Each transpose uses deterministic counting/transpose semantics; canonical family order is restored after arbitrary task completion.

Within-row strict-order validation is vectorized but semantically identical to a row-by-row predicate: every adjacent candidate index inside a CSR row must be strictly increasing.

Campaign MVIDX MUST NOT require a multi-billion-edge inverse payload and a full-family transpose workspace to coexist in anonymous RAM. Large-family inversion therefore supports bounded row-chunk CSR-to-CSC construction and file-backed NPY arrays under explicit RAM and disk admission. Candidate offsets remain canonical unsigned 64-bit arrays and candidate-to-witness indices remain canonical unsigned 32-bit arrays.

Chunk size, file-backing threshold, and concurrent inversion count are execution-only. Out-of-core and in-memory paths SHALL be byte-equivalent for the authoritative sparse arrays and content digest. Required disk capacity is preflighted before inversion. Durable authenticated state is re-opened before transient build paths are removed.

The producer/consumer driver SHALL respect bounded ready/in-flight/completed queue capacity even when the number of required families exceeds queue slots. It submits only admitted work, drains completions, and refills deterministically; it does not eager-submit an unbounded family set.

Exact reference-radius construction

TARGET-DATA2B reference-radius construction uses one shared read-only scaled matrix/tree per family and independent row blocks through the deterministic queue. Each queued cKDTree operation uses one native worker while outer work is available.

Execution may reduce a configured maximum block size to improve lane occupancy and bound query temporaries. Block boundaries are not scientific inputs; local radii and downstream reference arrays SHALL remain byte-identical across qualified block/worker schedules.

Pair/species lookup structures, constant-family rejection, and cached scaling may remove repeated object traversal or unnecessary computation only when inclusion rules and numerical results remain unchanged.

Exact sparse selector and qualification kernels

MVSEL/MVQUAL use typed ragged-CSR gathers and indexed reductions to replace repeated Python object traversal. Candidate/witness ordering remains canonical.

The MVSEL rank authority remains sequential. Vectorization MAY combine address calculation and telemetry work, but authoritative FP64 state mutations retain the historical candidate/edge order whenever exact equivalence depends on that order.

Required hard-obligation state and coverage counters may be maintained incrementally if qualification proves equality to reconstruction from the canonical sparse relation.

Same-N MVQUAL rescoring jobs are independent and may execute concurrently. Completion order is non-authoritative; comparison and persisted result order are reconstructed canonically. Campaign jobs use bounded native numerical lanes and memory admission to avoid nested oversubscription.

Deterministic repair execution

REPAIR retains the sequential repair iteration, objective, tie hierarchy, accepted/rejected trace, and winner application as authority. Immutable proposal scoring may execute concurrently when measured work exceeds the execution-only break-even threshold.

Proposal kernels may use vectorized CSR gathers, fused sparse scans, stamp-array membership, and O(1) candidate/rank maps. Parallel proposal completion is reduced in historical shortlist order. Before a winning swap is persisted, any arithmetic whose exact historical order is authoritative is recomputed in that order.

Selector-to-repair exact state reuse

`TargetMultiViewSelectionStateCache` is authenticated reconstructible state at the MVSEL/REPAIR boundary. MVSEL may snapshot exact mutable selector state at materializable rungs. REPAIR may restore such a checkpoint only while repair state is identical to the pure selector order.

After the first accepted repair swap, repair carries the historical mutable state forward. It SHALL NOT synthesize a later repaired state by reconciling a pure MVSEL checkpoint with selected-set differences when that operation changes FP64 state, even if selected candidate IDs happen to match.

Cache identity binds the reference/MVIDX/MVSEL/policy/sparse-kernel lineage and excludes worker/storage choices. Missing, stale, corrupt, or incompatible state falls back to exact historical replay. Post-divergence CSR gather preparation may be batched, but authoritative candidate-major FP64 mutations remain in canonical order.

Replay-source indexing and materialization

The selected replay ExtXYZ remains the external replay authority. A `ReplaySourceIndex` may record authenticated source-byte identity, frame byte offsets/lengths, atom counts, and source-order geometry identity so sparse monitor subsets can seek directly to requested frames and complete traversals can parse bounded contiguous chunks.

The index is reconstructible execution state and SHALL NOT replace replay source, split, label, prediction, or retention authority. Parser chunk size and index location are execution-only. Source mutation or index corruption causes safe reconstruction.

Parser concurrency is not introduced merely to increase worker count. It is permitted only when measured on the relevant workload and exact persisted replay bytes/identities are preserved.

Model inference, evaluation, and verification concurrency

Independent checkpoint-evaluation and bounded verification jobs may execute concurrently under common CPU/RAM/GPU/VRAM admission. Initialization/setup work is excluded or included in utilization calibration according to the current dedicated runtime specification; architecture requires only that the selected calibration policy be explicit, deterministic, and independent of scientific checkpoint metrics.

Runtime parallelism SHALL NOT enter evaluation policy, checkpoint metric, selection, or verification scientific digests. Existing completed verification/evaluation artifacts remain reusable only when their immutable model, structure/data, runtime dependency, and scientific execution identities remain compatible.

GPU admission SHALL fail closed on hard memory limits and SHALL NOT silently change backend/model precision or scientific policy. Positive accelerator qualification is evidence, not an architectural assumption.

Memory budget and persistence

CPU admission is necessary but insufficient. Long stages track an estimated memory budget including persistent trees/scaled arrays, in-flight temporaries, buffered completions, sparse state, result accumulation, and scratch space:

$$M_{\text{stage}} = M_{\text{persistent}} + M_{\text{inflight}} + M_{\text{buffered}} + M_{\text{sparse}} + M_{\text{result}} + M_{\text{scratch}}.$$

New work is admitted only when CPU and memory budgets permit it. Large reconstructible arrays may use mmap-compatible uncompressed persistence when that lowers peak memory or restart cost without changing scientific content.

Every persistent execution cache SHALL authenticate its semantic inputs and payload arrays independently. Cache corruption/staleness is a reconstruction event unless the cache itself is explicitly defined as scientific evidence by another contract.

NUMA-ready locality

A flat work queue is appropriate when memory locality is not limiting. Multi-socket systems may require node-local queues/data shards, worker affinity, local stealing first, and cross-node stealing only to avoid idle lanes.

NUMA behavior is an execution extension only. It SHALL be activated only after measurement on suitable hardware and SHALL NOT alter scientific identity or canonical reduction order.

Vectorization and allocation hygiene

Performance-critical loops SHOULD avoid repeated linear searches, rebuilding immutable dictionaries, repeated full-array scaling, unnecessary concatenation, Python-object materialization where contiguous typed arrays suffice, and per-frame/per-species mask reconstruction that can be cached safely.

Appropriate exact kernels include:

- offset-derived ragged CSR gathers;
- bounded-integer indexed counting/reduction;
- epoch/stamp arrays for bounded-ID membership;
- bounded batched bootstrap/statistical work that preserves the declared RNG stream;
- preallocated output arrays and static indexing metadata;
- cache-keyed static reduction metadata for repeated checkpoint evaluation.

Optimization changes must distinguish arithmetic preparation from authoritative arithmetic order. Rearranging addresses or batching independent work is acceptable only when the resulting authoritative records satisfy the applicable equivalence contract.

Progress and observability contract

Every long-running stage SHALL expose:

1. scientific progress: completed/total work, percent where meaningful, and ETA when estimable;
2. executor state: busy/allocated workers, ready/in-flight/buffered work, and resource pressure where measurable;
3. current hot items: identities/local progress for slow active families, shards, jobs, or proposals.

A heartbeat is emitted even when no task completes during the reporting interval. ETA is based on globally committed work rather than one current item.

User-facing MLFF progress uses the common presentation grammar:

- dynamic fields appear in canonical order beginning with status/progress/elapsed/ETA;
- elapsed and known ETA use fixed-width `HH:MM:SS`; durations beyond 99 hours retain all hour digits;

- unavailable ETA is exactly `--:--:--`;
- counted work uses `progress=completed/total (percent%)`;
- throughput carries an explicit stable unit;
- fields are semicolon-delimited;
- scheduler heartbeats expose completed and active/pending/queue state rather than prose-only status.

Presentation state SHALL NOT enter scientific digests or execution-cache identity. Shared timing/progress helpers own formatting so individual stages do not introduce private ETA dialects.

Performance qualification contract

A performance change is qualified against representative worker schedules and workloads appropriate to the stage. Qualification evidence records, as applicable:

- wall and CPU time;
- occupancy/utilization and throughput;
- peak RSS/VRAM and persisted bytes;
- queue occupancy/backpressure;
- output/content digests;
- exact scientific-record equality or the explicitly declared tolerance contract.

For sequential-authority algorithms, equivalence is checked at the state granularity needed to detect arithmetic drift: for example, MVSEL after each selected rank, REPAIR across the complete swap trace, and MVIDX across every canonical offset/index array.

Detailed measurements, historical before/after comparisons, rejected implementation experiments, and release-by-release optimization chronology belong in `benchmarks/`, `audits/`, `release/`, and `docs/history/mlff/`, not in this architecture chapter.

Part VII - Ownership boundaries and decision summary

Physical-observable validation ownership boundary

Physical observable calculation is not owned by `mdstats.training_data`. RDF, coordination, neighbor-angle statistics, connectivity, topology statistics, MSD, VACF, spectra, VDOS, diffusion, displacement distributions, current correlations, ionic conductivity, and related physical observables remain authoritative in their respective `mdstats.analysis` modules, specifications, and architecture manuals.

The MLFF layer owns only:

1. choosing an advisory observable-recommendation profile and an explicit recipe;
2. constructing an immutable recipe of analysis call IDs and parameters;
3. running the same recipe on matched reference and MLFF collections;
4. preserving verified collection/frame-selection identity, symmetric reference/candidate trajectory-generation identity, runtime/capability identity, warning records, and analysis-owned result identities;
5. binding execution to an explicit statistical role and, where required, a predeclared comparison policy, protocol freeze, and test-activation record;

6. applying comparison and acceptance policies only after those policies are frozen and independently identified.

It does not own physical numerical algorithms, normalization, neighbor definitions, plateau estimators, spectral transforms, or graph statistics.

The standardized analysis facade is `mdstats.analysis.observable_validation`. The MLFF-owned bridge delegates to that facade and stores no duplicate scientific arrays or competing result schemas.

Advisory recommendation profiles include generic condensed, crystalline-solid, amorphous-solid, liquid, and interface use cases. They are call-set recommendations, not automatic material classifiers. Users still supply applicable groups/species, cutoffs, projections, trajectory windows, thermodynamic conditions, and geometry-specific inputs. Ionic-transport and porous/zeolite/ring/cage/site analyses are explicit extensions and are never activated merely because a reference application uses those concepts.

Selection features versus validation observables

Compact structural descriptors used for partitioning or frame selection are MLFF workflow inputs. Full physical observables used to judge a trained model remain analysis products. An MLFF feature provider may call a lower-level analysis primitive only under that primitive's explicit contract and records the owner API; it cannot redefine the observable. Expensive trajectory observables such as diffusion, VDOS, conductivity, or residence statistics are validation jobs rather than ordinary frame-selection features.

Observable execution identity and evidence

Observable recipes validate declared dependencies before execution, preflight collection requirements, and bind versioned capability/codec identity. Supplied collection identities are recomputed and verified; location hints do not alter scientific identity. Reference and candidate trajectory-generation records bind their output collections symmetrically. Native analysis results receive analysis-owned identities; MLFF evidence stores those identities plus paired roles, warnings, durations, runtime identity, and upstream lineage.

Static equation-of-state, elasticity, finite-temperature thermomechanical response, viscosity, phonon, surface/interface, defect, and migration analyses remain owned by their dedicated analysis architecture/specification families.

Statistical role, policy ordering, and locked-test leakage

Physical-observable evidence has one explicit role such as `training_diagnostic`, `checkpoint_monitor`, `outer_validation`, `calibration`, `locked_test`, or `external_benchmark`. The role is not inferred from filenames or caller context.

The allowed dependency order is:

```
ObservableComparisonPolicy
    +
ObservableValidationActivationRecord
    +
Reference/Candidate Collection and Generation Identities
    -> ObservableValidationEvidence
```

```
-> ObservableComparisonResult
-> ObservableAcceptanceDecision
```

The reverse edge is forbidden. Realized observables must not be inspected to choose their own acceptance policy. Locked-test activation additionally requires the frozen training protocol, partition assignment, and explicit evaluation activation. Locked-test observable evidence cannot alter feature fitting, selection, training protocol, checkpoint selection, calibration policy, or acquisition.

Documentation and module ownership

Cross-cutting architecture defines ownership and data/control relationships. Detailed current behavior belongs in the corresponding module specifications under `docs/specs/`. A current module specification may strengthen a local contract but may not contradict the cross-cutting scientific invariants in this manual.

Proposed new module behavior, migrations, or developer sequencing is coordinated in `workplans/` until implemented and accepted. Completed implementation chronology belongs in history/release notes; qualification evidence belongs in audits/release evidence; performance evidence belongs in benchmarks.

Decision summary

The MLFF subsystem follows ten scientific rules.

1. **Independent evidence remains independent.** Cross-validation uses fresh models, nested checkpoint monitors, and evaluation folds that never control checkpoint choice.
2. **The complete training protocol is the comparison unit.** Replay, objective, checkpoint, exposure, backend, and other protocol-defining choices are part of comparison identity.
3. **Selection and E0 fitting are training-domain local.** Transforms, fitted metrics, selection, residual difficulty, and atomic-reference corrections do not inspect held-out evidence.
4. **Physical facts and workflow decisions are separate.** Occurrence, geometry, labels, policies, fitted products, and runtime realizations remain distinct record responsibilities.
5. **Data and deformation conventions are explicit.** Label domains, stress, energy channels, E0 limitations, and ASE cell-matrix conventions are declared and audited.
6. **Declared focus physics receives explicit coverage.** Profile events, atom-group environment quotas, group-resolved metrics, and rare transitions cannot be hidden by abundant host statistics; material-specific semantics are explicit optional specializations.
7. **Weights and exposure are audited.** Selection, property loss, head balance, and realized loader duplication are separate records.
8. **Locked tests are operationally sealed.** Activation requires frozen protocol and committee identities plus the applicable explicit activation decision.
9. **Replay and uncertainty policies are enforced.** Candidate checkpoints obey target/group/replay constraints, while calibration is bound to the actual final committee and declared applicability domain.
10. **Expansion is append-only by default.** Active-learning children inherit existing roles and add new cohorts without silently rewriting prior evidence unless a new evaluation lineage is explicitly created.

References

[1] I. Batatia, D. P. Kovacs, G. N. C. Simm, C. Ortner, and G. Csanyi, "MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields," *Advances in Neural Information Processing Systems* **35**, 11423-11436 (2022). DOI: 10.48550/arXiv.2206.07697.

[2] ACEsuit, "MACE descriptors," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/descriptors.html (accessed 2026-07-27).

[3] H. Flyvbjerg and H. G. Petersen, "Error Estimates on Averages of Correlated Data," *Journal of Chemical Physics* **91**, 461-466 (1989). DOI: 10.1063/1.457480.

[4] J. Racine, "Consistent Cross-Validatory Model-Selection for Dependent Data: hv-Block Cross-Validation," *Journal of Econometrics* **99**, 39-61 (2000). DOI: 10.1016/S0304-4076(00)00030-0.

[5] D. R. Roberts, V. Bahn, S. Ciuti, et al., "Cross-Validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure," *Ecography* **40**, 913-929 (2017). DOI: 10.1111/ecog.02881.

[6] J. D. Morrow, J. L. A. Gardner, and V. L. Deringer, "How to Validate Machine-Learned Interatomic Potentials," *Journal of Chemical Physics* **158**, 121501 (2023). DOI: 10.1063/5.0139611.

[7] VASP Software GmbH, "Smearing technique," VASP Wiki. Available at: https://vasp.at/wiki/Smearing_technique (accessed 2026-07-27).

[8] ACEsuit, "Training," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/training.html (accessed 2026-07-27).

[9] ACEsuit, `mace-torch` 0.3.16, Python Package Index, released 2026-05-10. Available at: https://pypi.org/project/mace-torch/0.3.16/ (accessed 2026-07-27).

[10] ACEsuit, `estimate_e0s_from_foundation`, MACE reference implementation, version-locked by the adapter at implementation time. Current source available at: https://github.com/ACEsuit/mace/blob/main/mace/data/utils.py (accessed 2026-07-27).

[11] ACEsuit, "Multihead Replay Finetuning," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/multihead_finetuning.html (accessed 2026-07-27).

[12] ACEsuit, "Multihead Training for MACE," MACE documentation. Available at: https://mace-docs.readthedocs.io/en/latest/guide/multihead_training.html (accessed 2026-07-27).

[13] C. Schran, K. Brezina, and O. Marsalek, "Committee Neural Network Potentials Control Generalization Errors and Enable Active Learning," *Journal of Chemical Physics* **153**, 104105 (2020). DOI: 10.1063/5.0016004.

[14] A. R. Tan, S. Urata, S. Goldman, J. C. B. Dietschreit, and R. Gomez-Bombarelli, "Single-Model Uncertainty Quantification in Neural Network Potentials Does Not Consistently Outperform Model Ensembles," *npj Computational Materials* **9**, 225 (2023). DOI: 10.1038/s41524-023-01180-8.

[15] I. Batatia, P. Benner, Y. Chiang, et al., "A Foundation Model for Atomistic Materials Chemistry," *Journal of Chemical Physics* **163**, 184110 (2025). DOI: 10.1063/5.0297006.

[16] M. Kulichenko, B. Nebgen, N. Lubbers, J. S. Smith, et al., "Data Generation for Machine Learning Interatomic Potentials and Beyond," *Chemical Reviews* **124**, 13681-13714 (2024). DOI: 10.1021/acs.chemrev.4c00572.

[17] ACEsuit, `mace.tools.train`, MACE version 0.3.16 source, especially the validation-head iteration and last-head checkpoint rule. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/mace/tools/train.py (accessed 2026-07-27).

[18] ACEsuit, `mace.cli.run_train`, MACE version 0.3.16 source, especially multi-head assembly, replay-ratio duplication, head ordering, and loader construction. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/mace/cli/run_train.py (accessed 2026-07-27).

[19] ACEsuit, `mace-torch` version 0.3.16 `setup.cfg`, complete runtime `install_requires` contract. Available at: https://github.com/ACEsuit/mace/blob/v0.3.16/setup.cfg (accessed 2026-07-28).

[20] e3nn developers, `e3nn` 0.4.4 package metadata and dependency contract, Python Package Index. Available at: https://pypi.org/project/e3nn/0.4.4/ (accessed 2026-07-28).

[21] R. Kern, "A Simple File Format for NumPy Arrays," NumPy Enhancement Proposal 1, 2007. Available at: https://numpy.org/doc/1.13/neps/npy-format.html (accessed 2026-08-15).

[22] NumPy developers, "numpy.load," NumPy reference documentation. Available at: https://numpy.org/doc/stable/reference/generated/numpy.load.html (accessed 2026-08-15).

[23] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.

[24] R. J. Hyndman and Y. Fan, "Sample Quantiles in Statistical Packages," *The American Statistician* **50**(4), 361–365 (1996). DOI: 10.1080/00031305.1996.10473566.

[25] J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching," *Communications of the ACM* **18**(9), 509–517 (1975). DOI: 10.1145/361002.361007.

[26] SciPy developers, "scipy.spatial.cKDTree," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.html (accessed 2026-08-15).

[27] T. F. Gonzalez, "Clustering to Minimize the Maximum Intercluster Distance," *Theoretical Computer Science* **38**, 293–306 (1985). DOI: 10.1016/0304-3975(85)90224-5.

[28] SciPy developers, "scipy.spatial.cKDTree.query," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query.html (accessed 2026-08-15).

[29] SciPy developers, "scipy.stats.wasserstein_distance," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wasserstein_distance.html (accessed 2026-08-15).

[30] K. Jamieson and A. Talwalkar, "Non-stochastic Best Arm Identification and Hyperparameter Optimization," *Proceedings of AISTATS*, PMLR 51:240–248, 2016. Available at: https://proceedings.mlr.press/v51/jamieson16.html.

[31] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research* **18**(185), 1–52 (2018). Available at: https://www.jmlr.org/papers/v18/16-558.html.

[32] R. D. Blumofe and C. E. Leiserson, "Scheduling Multithreaded Computations by Work Stealing," *Journal of the ACM* **46**(5), 720-748 (1999). DOI: 10.1145/324133.324234.

[33] SciPy developers, `scipy.spatial.cKDTree.query_ball_point`, SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query_ball_point.html (accessed 2026-08-17).

[34] SciPy developers, compressed sparse row/column matrix documentation, including `scipy.sparse.csr_matrix` and `scipy.sparse.csc_matrix`. Available at: https://docs.scipy.org/doc/scipy/reference/sparse.html (accessed 2026-08-17).

[35] `threadpoolctl` developers, "Python helpers to limit native thread pools," project documentation and source. Available at: https://github.com/joblib/threadpoolctl (accessed 2026-08-17).

[36] J. Deters, J. Wu, Y. Xu, and I.-T. A. Lee, "A NUMA-Aware Provably-Efficient Task-Parallel Platform Based on the Work-First Principle," arXiv:1806.11128 (2018). Available at: https://arxiv.org/abs/1806.11128.

[37] NumPy developers, `numpy.bincount`, NumPy reference documentation. Available at: https://numpy.org/doc/stable/reference/generated/numpy.bincount.html (accessed 2026-08-17).